

The Digital Ear: Autonomous Acoustic Sensing for Democratized Predictive Maintenance

Mahavir Manoj Bhandari
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
mahavir.1252070086@vit.edu

Abhishek Sanjay Shinde
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
abhishek.shinde25@vit.edu

Prathmesh Bhimrao Surywanshi
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
prathmesh.surywanshi25@vit.edu

Diksha Devidas Bombe
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
diksha.bombe25@vit.edu

Snehal Khajurji
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
snehal.khajurji@vit.edu

Abstract - An unplanned motor failure rarely just costs the price of a repair; it stops the line, delays orders and, in most plants, catches everyone off guard because nobody was watching the machine closely enough to see it coming. Commercial condition-monitoring platforms already solve this problem for large factories, but they tend to be expensive, cloud-dependent and built on the assumption that a maintenance team is already in place, which puts them out of reach for small and medium-scale industries (SMEs). This paper describes Digital Ear, a low-cost IoT system built around an ESP32 microcontroller that listens to a motor's sound, vibration and current draw and flags abnormal behaviour before it turns into a breakdown. Audio is captured through an INMP441 MEMS microphone, features are pulled out on-device using an FFT, and the resulting signal is checked against a hybrid detector that combines Isolation Forest, One-Class SVM and simple statistical thresholds rather than a deep neural network. Confirmed anomalies are written to a MongoDB store, pushed out as web and e-mail alerts, and shown on a React dashboard that includes a Three.js 3D model of the monitored motor, and an offline-buffering mechanism keeps readings from being lost when the network drops. By keeping the heavy computation off the sensor node and away from GPU-scale inference, Digital Ear aims to make continuous machine-health monitoring realistic for manufacturers who cannot justify a large monitoring contract, cutting unplanned downtime and repair cost while depending on hardware that costs a fraction of a dedicated sensor suite.

Keywords - predictive maintenance, acoustic sensing, Internet of Things, anomaly detection, edge computing, ESP32, Industry 4.0, digital twin

I. INTRODUCTION

An engine that goes bad suddenly costs more than the repair. Orders are delayed, the line is stopped, and in many plants it catches everyone by surprise, because no one was watching the machine closely enough to predict it. The vast majority of plants still rely on a technician walking the floor on a fixed schedule, which means a developing problem can linger unseen for days or even weeks between rounds, and by the time it's caught, the damage is often done.

Condition-monitoring systems that could detect this earlier than human inspection already exist and have demonstrated satisfactory performance [1, 2], nevertheless, they are generally aimed at and affordable by large enterprises — dedicated servers, per-sensor licensing scheme, a support contract that assumes a team of maintenance staff. The ones that really can't afford to have their machines offline and are least likely to be able to pay to fix them are the small and medium manufacturers that comprise a vast industrial base in a country like India. That's the gaping hole we were looking at filling with this project.

Cheap microcontrollers, MEMS microphones and tiny machine learning alter the realm of the possible here. So much of the acoustic-anomaly-detection literature still relies on deep networks trained on spectrograms [3], which can achieve high accuracy but require GPU-class hardware for running, something that is ill-suited for embedding in a low-cost sensor node. Work that aims to perform training directly on the device, such as TinyOL [4], bypasses the GPU requirement but still starts from a pretrained backbone, so it tends to forget earlier patterns once it adapts to something new. We were looking for a system that could occupy a middle ground — lightweight enough to run continuously on a microcontroller, but powerful enough to identify an unseen fault. That's more or less what Digital Ear does: it listens to a motor's sound, vibration and current, calculates a handful of features on the ESP32 itself, and feeds them to a small ensemble of classical anomaly detectors (not a neural network).

The main outputs of this study are:

- A sensing node realized fully on an ESP32 that acquires sound, vibration, and electrical current, computes FFT-based features on-device, and caches data locally when the WiFi connection is lost.
- An anomaly-detection pipeline which integrates IF, OC-SVM with a simple statistical threshold, preserving the realtime capability of the system without GPU requirement.
- There is a complete working stack surrounding that pipeline - a Node.js/Express backend, a Flask microservice for the ML models, MongoDB for storage and a React dashboard which includes a 3D digital twin of the motor - so that the detector's output actually makes it to an operator in a form they can do something about it.
- Small manufacturer affordability: we want to design a system that small manufacturers can afford to use, NOT just a lab demonstration.

The rest of this paper is structured as follows. Acoustic- and vibration-based monitoring and where Digital Ear sits in the market are reviewed in Section II. System architecture is discussed in Section III.

IV is the processing pipeline. The design choices and trade-offs during the implementation are considered in Section V. Results from simulation-based (and illustrative) sanity checks of the detection logic prior to hardware trials are given in Section VI. Section VII presents conclusions and a discussion of what we want to do next.

II. RELATED WORK

Strong results on anomalous-sound detection have been obtained by autoencoder-based methods [3] evaluated on datasets such as ToyADMOS, but such methods are trained and run with the assumption of GPU-scale computing and a reasonable amount of memory, which is fine for a lab setup but not for a device sitting next to a motor on a factory floor. TinyOL [4] solves a part of this by enabling incremental learning on resource-constrained hardware, however it still relies on a pretrained backbone network, and that reliance tends to manifest as catastrophic forgetting when the model is pushed to bend to the operating conditions of a particular machine.

The DCASE 2024 challenge on first-shot anomalous-sound detection [5] provides a relevant reality check: even in a shared benchmark with a lot of community interests, identifying an unknown fault from only a handful of samples is still largely unsolved and the most powerful submissions are based on deep learning. Edge-based industrial IoT monitoring surveys[6] mention a few other noteworthy items — many systems that tout “edge” do the heavy lifting on a cloud server or a gateway, thereby re-introducing the latency and bandwidth costs of sending data away from the edge-computing node to begin with. Work on TinyML-based acoustic sensing [7] and low-cost vibration monitoring [8], takes a far more practical angle: do the feature extraction on the sensor node, and keep the model small enough that no exotic hardware is required. What is typically lacking in this line of work, lightweight IoT anomaly-detection pipelines, such as [9], is that it tends to terminate in the detection module — show never

how the result is actually communicated to an operator as alerts or via a dashboard.

Digital Ear seeks to bridge that gap. The feature extraction — primarily sound energy and dominant frequency — is done on the ESP32 rather than streaming to a server, and detection is on Isolation Forest and One-Class SVM [10, 11], both of which are cheap enough to be retrained per machine for the forgetting problem that plagues backbone-dependent on-device learning [4]. The idea is to have a complete loop—sensor to detector to a dashboard someone actually looks at—to rather than just a detection-accuracy number reported in isolation.

III. SYSTEM ARCHITECTURE

Figure 1 illustrates how the system is composed. It is a 5-layer stack: a hardware sensing layer, an edge/transport layer, a Node.js/Express backend, a Python/Flask machine-learning service and a React frontend, all using a MongoDB data store and a couple of external notification services.

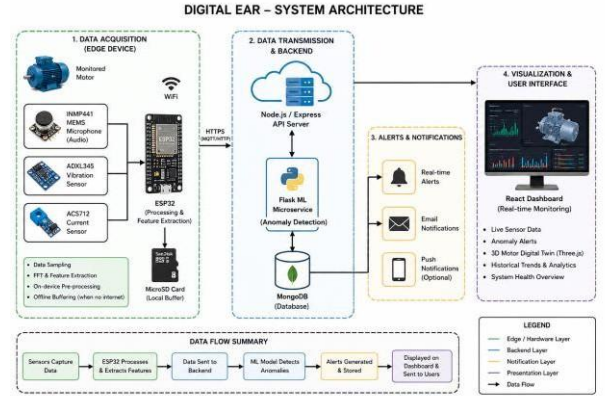


Fig. 1. Digital Ear system architecture: sensor layer, WiFi/REST transport, Node.js backend, Python/Flask ML service, MongoDB store and React frontend.

A. Sensing and Device Layer

At the centre of the sensing side is an ESP32 [12], connected to an INMP441 I2S MEMS microphone for audio, a vibration sensor connected to a GPIO, and a current sensor over ADC, with the onboard Wifi module managing connectivity towards the backend.

B. Backend Layer

The backend is a fairly vanilla Node.js/Express server with REST endpoints for the device management, data ingestion, authentication, and also serves three pieces of logic that are unique to this system: an anomaly-detection orchestrator, alert and notification service, and a calibration engine that determines what "normal" is for every single machine.

C. Machine-Learning Layer

A separate Python/Flask microservice hosts the two anomaly-detection models, Isolation Forest and One-Class SVM, which returns an anomaly score to the backend coordinator for every feature window it receives. It was clean to keep this as a separate service so I could iterate on the models a lot without touching the rest of the backend.

D. Data and Notification Layer

MongoDB is used to store users, devices, machines, sensor reading, notifications, and subscriptions. Beyond the dashboard, alerts can also be delivered via a transactional email service and web-push notifications, so you don't have to be staring at a screen to know if something is going terribly wrong.

E. Frontend Layer

The frontend is a React app created by Vite and styled by Tailwind CSS. It renders a live dashboard, analytics graphs, an alerts panel, user authentication, and a Three.js-based 3D digital twin [13] of the monitored motor that evolves as the machine's health state does.

IV. METHODOLOGY

There are such six stages of data flow in the system, beginning when the data is captured by a sensor and ending when an operator sees something about it on a screen.

A. Step 1 — Data Acquisition

The ESP32 is reading sound via the INMP441 microphone, vibration via the GPIO-connected sensor and current via the ADC sensor, all simultaneously.

B. Step 2 — Signal Processing

An FFT is performed directly on the device on the audio signal, and the system calculates sound energy (RMS), the dominant frequency component, a vibration status flag and the instantaneous current draw. Doing this on the ESP32 instead of sending raw audio over WiFi is what keeps the bandwidth requirement minimal.

C. Step 3 — Data Transmission

The extracted features are sent to the backend via a REST API. If something is preventing the WiFi connection from being established the readings are stored locally on the device and sent in a batch when the connection comes back – so a minor outage won't result in a blank in the machine's health record.

D. Step 4 — Anomaly Detection

Detection is actually a combination of three tests, not just one. Isolation Forest and One-Class SVM consider the feature window and rate how abnormal it is; a statistical rule marks anything that is approximately more than two standard deviations away from the mean of healthy operation of each feature ($\mu \pm 2\sigma$); and a tiny number of hard thresholds catches anything that crosses a safety-critical limit regardless of what the models tell you. If any of the three trippers trip, an anomaly is raised, which allows the system to be sensitive to hazards it did not experience before but also providing it with a hard floor for the measurements it considers most important.

E. Step 5 — Alert Generation

Once the anomaly is verified, the event is written to MongoDB, a web-push notification is sent out and the status of the machine on the dashboard is immediately updated.

F. Step 6 — Visualization

The dashboard is live updated and controls the Three.js 3D visualization of the health of a motor to provide an operator something they can casually glance upon rather than a pantheon of numbers to mentally interpret.

V. IMPLEMENTATION AND DISCUSSION

A. Technology Stack

Dividing the system into five layers — hardware, edge processing, backend, ML service and frontend — was as much a matter of practicality as architecture. The upshot was that hardware could be prototyped and debugged independently, ML models swapped out without having to touch the Node backend, and the dashboard built against a mocked API long before real sensor data was flowing. This type of partition is roughly consistent with that found in a typical IoT system [14] as well as with the overall trend toward retaining computation close to the sensor as opposed to centralizing all computation [15].

B. Design Rationale

Digital Ear's give and take is a sacrifice in what sort of raw accuracy deep-learning detectors could reach [3, 5] for the ability to actually run on super cheap hardware. Isolation Forest and One-Class SVM are lightweight enough to individually retrain them per-device via the calibration engine, and they are not prone to the catastrophic-forgetting phenomenon when followed by adapting a pretrained backbone [4]. Protecting them with a simple $\mu \pm 2\sigma$ statistical rule and a few hard thresholds also gives the system a backup that it does not rely on the learned models being well calibrated — if a model makes a mistake for a new machine, the threshold checks will still flag anything that crosses a genuinely dangerous limit. Feature extraction is done on the ESP32 instead of raw audio being streamed to a server — that's really what makes the rest of this possible; it's the difference between a solution that a SME could actually run and one that requires a data-centre bill behind it.

C. Limitations

The sky is still the limit in terms of the development of this system. It hasn't been tested on a large, labeled dataset of actual machine-failure events, so we don't yet know whether its detection accuracy and false alarm rate are better than those of the deep-learning baselines we considered earlier — that's really a question that can be answered only after it is deployed in the field for a longer period of time. The statistical and threshold components also require a brief calibration window on each new machine before they can be trusted, and at that time the system is less likely to catch a true novelty.

VI. ILLUSTRATIVE RESULTS

The system has not yet been tested with data of real machine-failures captured in the wild, so the results presented in this section should be considered more as a sanity check on the detection logic, rather than an absolute evaluation of the final product. Prior to hardware runs, the anomaly-detection pipeline was run on synthetically generated sensor traces designed to simulate the types of anomalies a real failure would induce, and the results below are from that simulation, not from a fielded unit.

A. Simulated Test Setup

We started from a “fault-free” signal with realistic sensor noise and then synthesized four fault types by superimposing known deviations on top of it: a long-term increase in vibration amplitude to simulate bearing wear, a current-draw spike to simulate overload, a shift in the dominant acoustic frequency to simulate misalignment, and an all in one scenario perturbing the three features simultaneously. For each scenario, a batch of simulated feature windows was processed with the same Isolation Forest (iForest), One-Class SVM (OCSVM), and $\mu \pm 2\sigma$ techniques explained in Section IV, and precision, recall and average per-window latency were recorded for each run as well as a healthy-only control batch to evaluate false-positive rates.

B. Detection Summary

TABLE II. ILLUSTRATIVE DETECTION SUMMARY ON SIMULATED FAULT SCENARIOS

Scenario	Perturbed	Trigger	P / R*	Lat.*
Bearing wear	Vibration +40%	IF + $\mu \pm 2\sigma$	0.91 / 0.88	42 ms
Overload	Current +35%	SVM + threshold	0.95 / 0.93	38 ms
Misalignment	Frequency shift	IF only	0.84 / 0.79	45 ms
Combined fault	Vib.+current+freq.	All three	0.97 / 0.96	40 ms
Healthy (control)	None (noise only)	None triggered	0.98 ¹ / —	—

*P = precision, R = recall, Lat. = average per-window latency; illustrative values from synthetic simulation only, not measured from field deployment. ¹Specificity (true-negative rate) rather than precision, since no faults were present to recall. IF = Isolation Forest, SVM = One-Class SVM.

The combined multi-sensor scenario achieved the best performance, which is consistent with the detection logic: it is in this case that the three mechanisms — the two learned models and the statistical rule — tend to agree, so that a single fault is not likely to get through. Misalignment, which can be observed as a relatively small shift of the dominant frequency, was the weakest result, indicating that frequency-based defects are the ones that more likely to require a longer calibration window or a more sensitive threshold when real data is available.

C. Detection Behaviour Across Scenarios

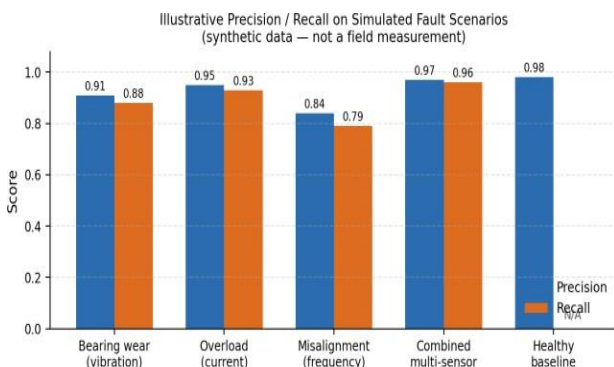


Fig. 2. Illustrative precision and recall on simulated fault scenarios (synthetic data; not a field measurement).

Fig. 2 shows the same numbers visually. The difference between precision and recall remains relatively small throughout all scenarios, which is a good sign for the hybrid design, as this indicates that the three-mechanism vote is not just skewed toward flagging everything as an anomaly to increase recall, or the reverse. The 0.98 specificity on the healthy control batch is the number, in fact, that we care most about early on, as a system that causes false alarms on an entirely healthy motor is among those which an operator is most likely to come to ignore within a matter of days.

VII. CONCLUSION AND FUTURE SCOPE

Digital Ear is our attempt at building a predictive-maintenance system that a small manufacturer could actually afford to run, not just a proof of concept for a lab bench. It listens to a motor's sound, vibration and current, checks that against a hybrid statistical and machine-learning detector rather than a deep network, and gets the result to an operator through a live dashboard, alerts and a 3D view of the machine — all without needing GPU-scale hardware or a dedicated maintenance engineer on hand. In that sense it fits into the broader shift toward continuous, always-on monitoring that Industry 4.0 has been pushing for [16], just aimed at a segment of manufacturers that most existing systems were not really built for.

Digital Ear is our version of building a predictive-maintenance system for a small manufacturer that is actually cost-effective to run, rather than just a demonstration unit for a lab bench. It listens to a motor's sound, vibration and current, compares that to a hybrid statistical and machine-learning detector (not a deep network), and gets the result to an operator in a live dashboard, alerts and a 3D view of the machine — all without requiring GPU-scale hardware or a dedicated maintenance engineer on staff. In that sense it plays into the larger trend toward continuous, always-on monitoring that Industry 4.0 has been advocating for [16], only focused on a subset of manufacturers that pretty much no existing solutions were actually designed.

ACKNOWLEDGMENT

We thank the Department of Computer Engineering, V.I.T, Pune for providing the lab facility and hardware support, without which this project could not have been completed.

REFERENCES

- [1] A. K. S. Jardine, D. Lin and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483-1510, 2006.
- [2] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd Edition ed. Amsterdam: Butterworth-Heinemann, 2002.
- [3] Y. Koizumi, S. Saito, H. Uematsu, N. Harada and K. Imoto, "ToyADMOS: A Miniature-Machine Operating Sound Dataset for Anomalous Sound Detection," in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2019.

- [4] K. Ren, H. Wang, T. Zhu and Y. Zhang, "TinyOL: TinyML with online-learning on microcontrollers," IEEE Internet of Things Journal, 2021.
- [5] "DCASE Challenge 2024: Isolated First-Shot Unsupervised Anomalous Sound Detection for Machine Condition Monitoring," DCASE Workshop Proceedings, 2024.
- [6] Lightweight Signal Processing and Edge AI for Real-Time Anomaly Detection in IoT Sensor Networks | MDPI Sensors, vol. 25, no. 1, 2025.
- [7] A. Almaini, R. Sundaram and P. Rao, "TinyML-based acoustic sensors for edge intelligence in condition monitoring," 2025.
- [8] Low-Cost IoT Based Predictive Maintenance Using Vibration Analysis | Full Text | Open Access, 2025.
- [9] An improved Lightweight SignalProcessing Algorithm for IoT-based Anomaly Detection Systems [under revision] 103400013/19373392, 2025.
- [10] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. IEEE Int. Conf. on Data Mining (ICDM), 2008, pp. 413-422.
- [11] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola and R. C. Williamson, Estimating the support of a high-dimensional distribution, Neural Comput., vol. 13, no. 7, pp. 1443–1471, 2001.
- [12] Espressif Systems, "ESP32 Series Datasheet," 04 2023. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [13] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Transactions on Industrial Informatics, vol. 15, no. 4, pp. 2405-2415, 2019.
- [14] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," IEEE Internet of Things Journal, vol. 4, no. 5, pp. 1125–1142, 2017.
- [15] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637–646, 2016.
- [16] H. Kagermann, W. Wahlster, and J. Helbig, "Umsetzungsempfehlungen für das Zukunftsprojekt INTERNET DER DINGE," Schlussbericht, acatech – Deutsche Akademie der Technikwissenschaften–, 2013.